7. OpenClaw Gateway Operation Manual

The Gateway is the resident process of OpenClaw: it maintains the connections of each messaging channel, hosts the control and event planes, and serves as the unified entry point for sessions, routing, and channel status. In simple terms: whether you can "connect to OpenClaw", whether the console can be opened, and whether messages can come in often depends on whether the Gateway is running normally.

7. OpenClaw Gateway Operation Manual

1. When You Need to Care About the Gateway
2. The Fastest Way to Start (Local)
3. What "External Interfaces" the Gateway Provides
4. Hot Reload and "Restart-Required" Changes
5. Authentication and Remote Access (Most Error-Prone)
    - 5.1 Gateway Authentication (token/password)
    - 5.2 Recommended Way for Remote Access: Tunnel First, Then Access
    - 5.3 Console Cannot Be Opened over Plain HTTP (device identity required)
6. Multiple Instances and "Rescue Robot" Mode
7. Troubleshooting Order, Which Locates Most Problems
    - 7.1 Check the Status First
    - 7.2 Gateway Will Not Start: Configuration Validation Failure
    - 7.3 Common Symptom: Service Installed but Not Actually Running

1. When You Need to Care About the Gateway

In most cases, you only need to keep it running stably; you only need to specifically check/learn about the Gateway in the following scenarios:

- Opening the console in a browser on the Jetson often reports errors, and you are preparing to switch to the TUI or remote access
- The configuration has been changed but "it feels like it has not taken effect", and you suspect that the hot reload/restart was not triggered
- You want to expose the console to the LAN or access it remotely via SSH/Tailscale
- You want to do multi-instance isolation (for example, a "rescue robot"/redundancy)
- Port conflicts, the process will not start, or the service "looks installed but is not running"

2. The Fastest Way to Start (Local)

Execute the following on the gateway host (for example, the Jetson):

```
openclaw gateway
```

Common parameters:

```
# Print more complete debugging/tracing information to the current terminal
(useful for troubleshooting)
openclaw gateway --port 18789 --verbose

# When the port is occupied, try to terminate the listener occupying the port and
force startup
openclaw gateway --port 18789 --force
```

Port priority (from highest to lowest): `--port` > `OPENCLAW_GATEWAY_PORT` > `gateway.port` > the default `18789`.

3. What "External Interfaces" the Gateway Provides

The same port (default 18789) provides both the WebSocket control plane and HTTP services (control interface, hooks, A2UI, etc.) at the same time, which is "single-port multiplexing".

Points you will commonly use:

- Local console: `http://127.0.0.1:18789/`
- OpenAI Chat Completions compatible interface: `/v1/chat/completions`
- OpenResponses interface: `/v1/responses`
- Tool invocation interface: `/tools/invoke`

In addition, by default the gateway also starts a Canvas static file service (default port 18793) to provide editable HTML/A2UI resources (served from `~/.openclaw/workspace/canvas` by default). To disable it, set `canvasHost.enabled=false` or `OPENCLAW_SKIP_CANVAS_HOST=1`.

4. Hot Reload and "Restart-Required" Changes

The gateway monitors `~/.openclaw/openclaw.json` (or the path specified by `OPENCLAW_CONFIG_PATH`), and configuration updates are **usually applied automatically**. The default reload mode is `gateway.reload.mode="hybrid"`: safe changes are hot-applied, and critical changes trigger a restart.

You can explicitly configure the reload behavior:

```
{
  "gateway": {
    "reload": { "mode": "hybrid", "debounceMs": 300 }
  }
}
```

Experience-based suggestions:

- Changing business configurations such as models, agents, and routing usually does not require you to restart manually
- Changing `gateway.*` (port, binding, authentication, TLS, HTTP, etc.) counts as an infrastructure change and is more likely to trigger a restart

5. Authentication and Remote Access (Most Error-Prone)

5.1 Gateway Authentication (token/password)

Gateway authentication is enabled by default: you can set `gateway.auth.token` (or `OPENCLAW_GATEWAY_TOKEN`) or `gateway.auth.password`. When connecting, the client needs to carry it in `connect.params.auth.token/password`.

If you use the wizard flow, a token is usually generated by default (even if it is bound only to loopback).

5.2 Recommended Way for Remote Access: Tunnel First, Then Access

Most recommended: Tailscale/VPN. Next: SSH tunnel.

Example (mapping remote 18789 to local 18789):

```
ssh -N -L 18789:127.0.0.1:18789 user@host
```

Then you access it locally:

- Web: `http://127.0.0.1:18789/`
- WS: `ws://127.0.0.1:18789`

Note: even through a tunnel, if the gateway is configured with a token, the client still has to carry the token to connect.

5.3 Console Cannot Be Opened over Plain HTTP (device identity required)

If you open the console on the LAN with `http://<lan-ip>:18789/`, the browser may be in a non-secure context, which restricts WebCrypto and prevents the device identity from being generated, resulting in `device identity required` / `connect failed`.[^gateway_troubleshooting]

Preferred fix route:

- Open locally: `http://127.0.0.1:18789/`
- For remote scenarios, use Tailscale Serve to provide HTTPS
- If HTTP must be used, enable `gateway.controlUi.allowInsecureAuth: true` and use the gateway token (token mode only, without device identity/pairing)[^gateway_troubleshooting]

6. Multiple Instances and "Rescue Robot" Mode

> Try this operation only when needed

Usually running one gateway per host is enough; running multiple gateway instances is recommended only when redundancy or strong isolation (for example, a rescue robot) is required.

The core principle for multiple instances is "full isolation + no conflicts":

- Different `gateway.port`

- Different `OPENCLAW_CONFIG_PATH`
- Different `OPENCLAW_STATE_DIR`
- Different workspace (`agents.defaults.workspace`)

Under the dev profile, you can quickly start a fully isolated development instance without affecting the main environment:

```
openclaw --dev setup
openclaw --dev gateway --allow-unconfigured
openclaw --dev status
openclaw --dev health
```

7. Troubleshooting Order, Which Locates Most Problems

7.1 Check the Status First

Execute in order:[^gateway_troubleshooting]

```
openclaw status
openclaw status --all
openclaw status --deep
```

Common supplementary commands:

```
openclaw gateway probe
openclaw channels status --probe
openclaw gateway status
openclaw logs --follow
openclaw doctor
```

7.2 Gateway Will Not Start: Configuration Validation Failure

OpenClaw configuration uses strict schema validation: unknown keys, type errors, or invalid values may all cause the gateway to refuse to start.

When validation fails, usually only diagnostic commands are available (for example, `openclaw doctor / logs / health / status`). It is recommended to directly run:

```
openclaw doctor
```

7.3 Common Symptom: Service Installed but Not Actually Running

If you install the gateway as a service with systemd/launchd/schtasks, showing "loaded/installed" does not mean the process is running. Check first:[^gateway_troubleshooting]

```
openclaw gateway status
openclaw logs --follow
```